- Keep your smart contract code simple, coherent, and auditable. Let each state variable and function specified in a smart contract address a single problem. Do not include redundant data or unrelated functions. Make the smart contract functions auditable by using custom function modifiers instead of inline (`if/else`) code for checking pre- and post-conditions of a function's execution.
- Use function access modifiers for
    - Implementing rules, policies, and regulations for data access for all the participants
    - Implementing common rules for all who may access a function
    - Declaratively validating application-specific conditions
    - Providing auditable elements to allow verification of the correctness of a smart contract
- Use the `memory` type as a qualifier for local variables that don't need to be stored on the blockchain. Memory variables are transient and are not stored. (You saw an example in listing 3.7.)
- Develop the smart contract in incremental steps, debugging each step.
- Be aware that the Solidity language updates frequently to improve performance and security. In this case, you have to adjust your code to meet the requirements of the latest version.

3.7 Retrospective

The design process you've learned in this chapter--creating a use case diagram; identifying users, data assets, and FSM state transitions; creating a contract diagram; and writing smart contract code--enables you to analyze a problem systematically